ÇÖZÜM TASARIM DÖKÜMANI

(SOLITION DESIGN DOCUMENT – SDD)

PROJE ADI

YAPAY ZEKA DESTEKLİ TASARIM ÜRETİM VE YÜKLEME SÜRECİ

HAZIRLAYAN

OĞUZHAN EVCİ

TARİH

19 MART 2026

İçindekiler

İçindekiler 1
1. Proje Özeti (Executive Summary) 2
2. Mevcut Durum ve Hedeflenen Durum (As-Is vs. To-Be) 3
3. Önkoşullar ve Kapsam Dışı Durumlar (Prerequisites & Out of Scope) 4
A. Önkoşullar (Prerequisites) 4
B. Kapsam Dışı Durumlar (Out of Scope) 5
4. Kullanılan Sistemler ve Uygulamalar (In-Scope Applications) 5
5. Girdi ve Çıktılar (Inputs & Outputs) 6
A. Süreç Girdileri (Inputs) 6
B. Süreç Çıktıları (Outputs) 7
6. Klasör ve Veri Yapısı (Directory Structure) 7
A. İşlem Klasörleri (Processing Folders) 7
B. Kök Dizin Dosyaları (Root Directory Files) 8
7. Süreç Akışı ve Sistem Mimarisi (Process Architecture & Workflow) 8
7.0. Global İstisna Yönetimi ve Durum Geçişleri (State Machine & Transitions) 8
7.1. Başlangıç ve Veri Hazırlığı (Initialization State) 9
7.2. Faz 1: Otonom Üretim ve Optimizasyon Döngüsü (Faz1_IslemDongusu.xaml) 10
A. Veri Okuma ve Oturum Kontrolü (Giriş Adımları) 10
B. Ana İşlem Döngüsü ve Statü Kontrolü (Main Loop & Status Validation) 11
C. Sayfa Doğrulama ve Parametre Hazırlığı (Seq: 3 : 4 Boyut Kontrolü) 12
D. Dinamik Boyut Yönetimi ve Veri Hazırlığı (Aspect Ratio & Data Preparation) 12
E. Yapay Zeka Üretimi ve Dinamik Bekleme (Seq: Prompt Girişi ve Üretim) 13
F. Hafıza Yönetimi ve Yeni Döngü Hazırlığı (Seq: Hafıza ve Liste Temizliği) 14
G. Kullanıcı Onayı ve Tasarım Seçimi (MsgBox: Manuel Görsel İndirme) 14
H. AI Destekli SEO Analizi ve RegEx Veri Ayıklama (Seq: Gemini Analizi ve RegEx ile Karar Çekme) 15
I. SEO Kayıt ve Dosya Transferi (Seq: SEO: İsimlendirme ve Dosya Taşıma) 16
J. Statü Senkronizasyonu ve Python Ön Hazırlık (Seq: Python Hazırlık) 17
K. Gelişmiş Görüntü İşleme ve Koşullu Durum Yönetimi (Seq: Python Tasarım İşleme Modülü) 17
7.3. Finalize_Workflow: Süreç Kapanış ve Raporlama (Finalize_Workflow.xaml) 19

A. Kapanış Başlangıcı ve Durum Analizi (Log: Final Başladı & If: Hata Kontrolü) 19
B. Akıllı Hata Çözümleme ve Doğal Dil Dönüşümü (Seq: Gemini AI Hata Analizi) 19
C. Hata Bildirimi ve Otomatik E-Posta Gönderimi (Seq: Hata Mesajı Gönder) 20
D. Başarı Bildirimi ve Süreç Tamamlama (Seq: Başarı Mesajı Gönder) 21
E. Süreç Sonu ve Arşiv Yönetimi (Seq: Süreç Sonu ve Arşiv İşlemleri) 22
F. Sistem Temizliği ve Operasyonel Kapanış (Kill Process & Final Log) 23
8. Hata ve İstisna Yönetimi (Exception Handling) 24
A. Global İstisna Yakalama Mekanizması 24
B. Kritik Müdahale ve Throw Kullanımı 24
C. Hata Sonrası Durum Geçişleri (Transitions) 24
9. Süreç İzlenebilirliği ve Loglama Stratejisi (Logging & Traceability) 25
A. Stratejik Dar Boğaz ve Kontrol Noktası Loglaması (Checkpoint Logging) 25
B. Log Seviyeleri ve Kullanım Amaçları 25
C. Arşivleme ve Geriye Dönük Raporlama 25
10. Veri Sözlüğü ve Parametre Yönetimi (Data Dictionary & Parameters) 26
A. Kritik Argümanlar ve Modüler Veri Transferi 26
B. İsimlendirme Standartları (Naming Conventions) 26

1. Proje Özeti (Executive Summary)

Bu proje, **Print-on-Demand (POD)** iş modeliyle çalışan e-ticaret mağazaları için tasarım üretim, optimizasyon ve raporlama süreçlerini uçtan uca otomatize etmeyi amaçlamaktadır. Özellikle öğretmen meslek grubuna yönelik (niche) tasarımların seri üretimine odaklanan bu RPA robotu; manuel iş yükünü ortadan kaldırarak hızlı, SEO uyumlu ve baskıya hazır (print-ready) ürünler elde edilmesini sağlar.

Geliştirilen robot, öncelikle Excel'de bulunan taslak bir tasarım promptunu farklı branşlardaki öğretmen gruplarına göre dinamik olarak uyarlar. Ideogram platformunda yapay zeka ile görsel üretimi tetiklendikten sonra robot süreci duraklatarak kullanıcıya bir onay ekranı (Human-in-the-Loop) sunar. Kullanıcının üretilen alternatifler arasından en uygun tasarımı seçip bilgisayara kaydetmesiyle süreç otonom olarak devam eder.

Kaydedilen ham tasarım, Google Gemini AI tarafından analiz edilerek algoritma dostu ve SEO kurallarına uygun şekilde otomatik isimlendirilir. Ardından POD platformlarının (Redbubble vb.) zorunlu baskı standartlarını sağlamak amacıyla Python görüntü işleme modülü devreye girer. Bu modül, görselin arka planını tamamen transparan hale getirir ve çözünürlüğünü yüksek kaliteli baskı için 300 DPI seviyesine çıkarır.

Tüm liste tamamlandığında, ürünler POD mağazasına yüklenmeye hazır hale gelir ve kullanıcıya bir başarı raporu e-postası iletilir. Herhangi bir teknik hata durumunda ise State Machine (Durum Makinesi) devreye girip hatayı Gemini AI ile kullanıcı dostu bir metne çevirerek anında uyarı mesajı atar. İşlem bitiminde güncel takip tablosu arşivlenir ve sistem güvenli bir şekilde kapatılır.

2. Mevcut Durum ve Hedeflenen Durum (As-Is vs. To-Be)

Mevcut Durum (As-Is): Mevcut işleyişte, Print-on-Demand (POD) platformları için tasarım üretim ve hazırlık süreçleri tamamen manuel olarak yürütülmektedir. Şablon tasarım komutlarının (prompt) farklı nişlere göre tek tek uyarlanması, Ideogram platformuna girilerek üretimin beklenmesi ve görsellerin manuel olarak kaydedilmesi ciddi zaman almaktadır. Ayrıca, kaydedilen ham görsellerin arama algoritmalarına (SEO) uygun şekilde yeniden isimlendirilmesi ve baskı standartlarına (şeffaf arka plan, 300 DPI) getirilmesi için her görsele ayrı ayrı işlem yapılması gerekmektedir. Çok sayıda tasarım için bu adımların tekrar edilmesi, hem süreci yavaşlatmakta hem de isimlendirme ve kalite standartlarının korunmasını zorlaştırmaktadır.

Hedeflenen Durum (To-Be): Geliştirilen RPA süreci ile manuel müdahale sadece kalite kontrol (tasarım seçimi) aşamasına indirgenmiştir. Ana Excel listesindeki şablon promptların farklı nişlere dinamik olarak uyarlanması, onlarca farklı tasarımın otonom şekilde üretilmesi ve Google Gemini AI ile sıfır hatalı SEO isimlendirmesi yapılması hedeflenmektedir. Görüntü işleme aşamasında devreye giren Python scriptleri (optimizasyon modülleri) sayesinde, yapay zekanın ürettiği orijinal tasarım kalitesi hiçbir kayba uğramadan birebir korunur ve şeffaf arka plan/300 DPI standartlarında baskıya tam hazır hale getirilir. Bu sayede tasarım başı harcanan sürenin %90 oranında kısaltılması, profesyonel bir kalite güvencesi oluşturulması ve ürünlerin ilgili pazaryerlerine (Redbubble vb.) **daha hızlı yüklenerek satışa sunulması sağlanacaktır.**

3. Önkoşullar ve Kapsam Dışı Durumlar (Prerequisites & Out of Scope)

Robotun kararlı çalışması ve modüller arasındaki entegrasyonun bozulmaması için aşağıdaki teknik şartların sağlanması zorunludur.

A. Önkoşullar (Prerequisites)

- **Python Çalışma Ortamı:** Sistemde Python 3.10+ sürümü yüklü olmalıdır.
- **Gerekli Kütüphaneler:** Görüntü işleme motorunun çalışabilmesi için rembg, Pillow (PIL), io ve os kütüphanelerinin terminal üzerinden yüklenmiş olması gerekir.
- **Yapay Zeka Modeli:** Arka plan silme işlemi için birefnet-general modelinin sisteme indirilmiş veya internet erişimiyle erişilebilir olması şarttır.
- **API ve Oturumlar:** Gemini API kotalarının aktif olması ve Ideogram platformunda üretim yapabilecek yeterli kredinin bulunması zorunludur.

B. Kapsam Dışı Durumlar (Out of Scope)

- **Kreatif Prompt Yazımı:** Ana şablonda yer alacak "Master Prompt" metinlerinin sıfırdan kurgulanması robotun görevi değildir.
- **Finansal Süreçler:** Kredi bitimi durumunda otomatik ödeme yapılması veya paket satın alınması kapsam dışındadır.

4. Kullanılan Sistemler ve Uygulamalar (In-Scope Applications)

Süreç boyunca robotun veri alışverişinde bulunduğu ve otonom olarak etkileşime girdiği temel uygulama ve servisler şunlardır:

- **UiPath Studio:** Sürecin ana orkestrasyonunu üstlenen, **State Machine (Durum Makinesi)** mimarisiyle tasarlanmış ve tüm iş akışının yönetildiği geliştirme ortamıdır.
- **Google Chrome:** Ideogram.ai platformu üzerinden görsel üretim süreçlerinin ve arayüz etkileşimlerinin yürütüldüğü ana tarayıcıdır.
- **UiPath Integration Service (Google Gemini Entegrasyonu):** Görsel analizi, SEO uyumlu isimlendirme ve hata mesajlarının sadeleştirilmesi için kullanılan yapay zeka servisidir. Bağlantı, doğrudan API Key yerine daha güvenli ve kurumsal bir yapı olan **UiPath GenAI Activities** entegrasyonu ile sağlanmıştır.
- **UiPath Computer Vision (AI Service):** Ideogram arayüzündeki dinamik butonların (Generate vb.) ve parametrelerin tespiti için kullanılan yapay zeka destekli görüntüleme servisidir. Bu servis için **UiPath API Key** doğrulaması kullanılmıştır.
- **Python (Görüntü İşleme Motoru):** Yapay zeka tarafından üretilen ham görsellerin baskı standartlarına (300 DPI, Arka Plan Silme) getirilmesini sağlayan design_production_tool.py scriptinin çalışma ortamıdır. Bu modül, arka plan temizleme işlemi için **birefnet-general** yapay zeka modelini kullanır [python script source 1].
- **MS Excel:** Taslak promptların, niş verilerinin, işlem statülerinin ve süreç sonu arşiv raporlamalarının tutulduğu ana veri yönetim merkezidir.
- **Gmail:** Süreç başarıyla tamamlandığında veya bir hata oluştuğunda kullanıcıya Google Integration Service üzerinden anlık bildirim gönderilmesini sağlayan e-posta servisidir.

5. Girdi ve Çıktılar (Inputs & Outputs)

A. Süreç Girdileri (Inputs)

- **Ana Tasarım Şablonu (01-Templates):** Sürecin temelini oluşturan, içerisinde dinamik etiketlerin ({NICHE}, {OBJECTS}) bulunduğu ana Excel dosyasıdır.
- **Yapılandırma Dosyası (Config.xlsx):** Robotun çalışma anında ihtiyaç duyduğu dosya yolları, e-posta adresleri ve sistem ayarlarını barındıran merkezi kontrol dosyasıdır.
- **Görüntü İşleme Betiği (design_production_tool.py):** Yapay zeka çıktılarını baskı standartlarına dönüştüren ana Python kod dosyasıdır.
- **AI Prompt Şablonları (Stratejik Girdi):** Gemini Vision modülüne gönderilen SEO analiz talimatları ve hata yönetiminde kullanılan "RPA Asistanı" rol tanımlamalarıdır.
- **Kullanıcı Seçimi ve Ham Görsel (Intermediate Input):** Ideogram tarafından üretilen alternatifler arasından kullanıcının onaylayıp 03-Gemini-Temp klasörüne kaydettiği yüksek kaliteli görseldir .
- **birefnet-general Model Verisi:** Python modülünün arka plan silme işlemi sırasında RAM üzerinde kullandığı yapay zeka model ağırlıklarıdır.

B. Süreç Çıktıları (Outputs)

- **Baskıya Hazır Tasarımlar (05-Final-Process):** Python modülü tarafından arka planı temizlenmiş, **300 DPI** çözünürlüğe getirilmiş ve **4500x5400 px** boyutlarında optimize edilmiş nihai PNG dosyalarıdır.
- **SEO Optimizasyonlu Ara Dosyalar (04-Optimize-Process):** Gemini tarafından analiz edilip RegEx ile ayıklandıktan sonra SEO kurallarına göre yeniden isimlendirilmiş PNG dosyalarıdır .
- **AI Üretimi SEO Metadataları:** Gemini Vision tarafından üretilen; tasarımın tarzını doğrulayan analiz metinleri ve SEO kurallarına uygun dinamik dosya ismi önerileridir.
- **Sadeleştirilmiş Hata Raporları:** Teknik jargondan arındırılmış, Gemini AI tarafından hazırlanan ve kullanıcıya hatanın nedenini açıklayan "insancıl" bilgilendirme metinleridir.
- **Arşivlenmiş İş Raporu (06-Reports-Archive):** Sürecin hangi aşamada olduğunu, SEO isimlerini ve işlem statülerini içeren, zaman damgalı nihai Excel takip dosyasıdır .
- **Süreç Bildirimleri (Gmail):** Operasyonun durumuna göre paydaşlara iletilen başarı veya hata bildirim e-postalarıdır.

6. Klasör ve Veri Yapısı (Directory Structure)

Sürecin veri bütünlüğünü sağlamak, dosyaları düzenli takip etmek ve RPA modülleri ile Python scripti arasındaki veri alışverişini yönetmek amacıyla aşağıdaki standart dizin yapısı kullanılmaktadır. Sistem; işlem klasörleri ve kök dizin dosyaları olmak üzere iki ana gruba ayrılmıştır:

A. İşlem Klasörleri (Processing Folders)

- **01-Templates:** Sürecin başlangıcında kullanılacak orijinal ana şablon Excel dosyasını barındırır. Robot, ana verinin (master data) bozulmamasını garanti altına almak için her sürecin başında bu şablonu kopyalayarak işleme başlar.
- **02-Work-In-Progress:** 01-Templates klasöründen kopyalanan çalışma dosyasının (WIP) bulunduğu ana operasyon klasörüdür. Robot; promptların niş gruplara uyarlanması, işlem adımları ve güncel statü loglarını ("Done", "Hata" vb.) anlık olarak bu dosya üzerine yazar.
- **03-Gemini-Temp:** Ideogram üzerinden üretilip kullanıcının onayından geçen (Human-in-the-loop) ham tasarımın ilk olarak kaydedildiği geçici klasördür.
- **04-Optimize-Process:** Gemini AI tarafından SEO kurallarına uygun dosya adı verilmiş görselin taşındığı klasördür.
- **05-Final-Process:** Python tarafından başarıyla işlenmiş, arka planı temizlenmiş ve POD platformlarına yüklenmeye tam hazır (300 DPI) hale gelmiş nihai tasarımların tutulduğu klasördür.
- **06-Reports-Archive:** Süreç tamamlandığında, 02-Work-In-Progress klasöründeki güncel ve tamamlanmış ana takip tablosunun, o günün tarihi (zaman damgası) eklenerek güvenli bir şekilde yedeklendiği arşiv klasörüdür.

B. Kök Dizin Dosyaları (Root Directory Files)

Klasör yapısının ana dizininde (root), sürecin temel ayarlarını ve optimizasyon motorunu barındıran iki kritik dosya konumlandırılmıştır:

- **Config.xlsx:** Robotun sistem yollarını, API kotalarını ve çalışma anında ihtiyaç duyduğu tüm statik ayarları merkezi olarak tutan yapılandırma dosyasıdır.
- **design_production_tool.py:** 04-Optimize-Process klasörüne gelen görsellerin arka plan silme ve baskı standartlarına (300 DPI) getirilme işlemlerini sağlayan ana Python kod dosyasıdır.

7. Süreç Akışı ve Sistem Mimarisi (Process Architecture & Workflow)

Süreç, robotun kararlılığını (stability) artırmak, modülerliği sağlamak ve hata yönetimini (Exception Handling) merkezileştirmek amacıyla Main.xaml üzerinde bir **State Machine (Durum Makinesi)** mimarisiyle tasarlanmıştır.

7.0. Global İstisna Yönetimi ve Durum Geçişleri (State Machine & Transitions)

Sistem, beklenmedik anlarda çökmemesi için tüm ana adımları (State) Try/Catch blokları içerisinde yürütür. Alınan hatalar karakterine göre ayrıştırılarak global değişkenlere kaydedilir ve sürecin rotası (Transitions) bu değişkenlerin durumuna göre otonom olarak belirlenir:

- **Hata Sınıflandırması ve Kayıt Altına Alma:**
    - **System Exception:** Sayfanın yüklenmemesi, Ideogram'ın yanıt vermemesi gibi teknik kaynaklı hatalardır. Yakalandığında SystemException değişkenine kaydedilir.
    - **Business Rule Exception (BRE):** Şablon dosyanın bulunamaması, {NICHE} veya {OBJECTS} etiketlerinin eksik olması gibi veri ve iş kuralı ihlalleridir. Yakalandığında BusinessException değişkenine kaydedilir.
- **Durum Geçişleri (Transitions):**
    - **T1 (Başarılı Hazırlık):** *Initialization* aşaması bittiğinde hata değişkenleri boşsa (Nothing), sistem üretim aşaması olan Faz 1'e geçer.
    - **T2 (Başarılı Kapanış):** Faz 1 tüm liste için sorunsuz tamamlandığında sistem Final State'e geçer.
    - **T3 (Kritik Hata Yönlendirmesi):** *Initialization* veya *Faz 1* aşamalarının herhangi birinde yukarıdaki hata değişkenlerine bir veri kaydedilirse; robot o anki işlemi derhal keser ve T3 yolu üzerinden doğrudan **Final State (Kapanış)** evresine atlar.
- **Final State Entegrasyonu:** T3 yoluyla kapanışa gelen robot, hafızasındaki bu hata değişkenini doğrudan Google Gemini AI modülüne aktarır. Gemini, bu karmaşık logu son kullanıcının anlayacağı sade bir metne çevirerek e-posta yoluyla yetkiliye bildirir ve kapanış adımlarına geçilir.

7.1. Başlangıç ve Veri Hazırlığı (Initialization State)

- **Amacı:** Sistemin statik ayarlarını yüklemek, çalışma dosyalarını güvence altına almak ve taslak tasarım komutlarını (prompt) farklı nişlere göre çoğaltarak ana iş listesini (WIP) üretime hazır hale getirmektir. Tüm adımlar hata yakalama (Try/Catch) bloğu içinde yürütülür.
- **İş Kuralları ve İlgili Modüller:**
    - **Konfigürasyonun Yüklenmesi (Seq: Config Hazırlama):** Robot, sistem yollarını ve API kotalarını dinamik tutmak için yeni bir sözlük (Dictionary) nesnesi başlatır. Kök dizindeki Config.xlsx dosyası okunur ve içindeki tüm "Name-Value" (Anahtar-Değer) eşleşmeleri, satırlar döngüye alınarak robotun global hafızasına (**Config** değişkenine) aktarılır.
    - **Sistem Hazırlığı (Seq: Hazırlık: Ortam):** Arka planda açık olan Excel uygulamaları (dosya kilitlenme çakışmasını önlemek için) kapatılır ve orijinal şablon dosyasının varlığı (**bool_SablonVarMi** değişkeni ile) kontrol edilir.
    - **Şablon Doğrulama ve Kopyalama (FlowDecision: Şablon Var mı? & Seq: Hazırlık: Şablon & Veri):** Şablon dosya yoksa robot Business Rule Exception (BRE: Şablon Yok) fırlatarak T3 yoluna girer. Dosya varsa, veri bozulmasını önlemek için 02-Work-In-Progress klasörüne çalışma dosyası (WIP) olarak kopyalanır.
    - **Etiket Kontrolü (FlowDecision: Kontrol: Şablon Etiket):** Kopyalanan dosyadan ana tasarım komutu okunarak hafızaya alınır (**str_AnaKalip**). Metin içinde {NICHE} ve {OBJECTS} dinamik etiketleri bulunamazsa, süreç başlatılmadan hata fırlatılır (BRE: Etiket Hatası) ve sistem durdurulur.
    - **Liste İşleme Merkezi (Seq: LİSTE İŞLEME MERKEZİ):** Etiketler başarılıysa, ana iş listesi bir veri tablosu olarak (**dt_IsListesi**) okunur ve döngüye alınır. Statüsü "Pending" olan her satır için; şablon metindeki etiketler o satırın kendi niş verileriyle doldurulur (**str_YeniPrompt**). Yeni prompt Excel'e yazılır ve satır statüsü üretim sırasının geldiğini belirten **"ready_ceo"** olarak güncellenir.

7.2. Faz 1: Otonom Üretim ve Optimizasyon Döngüsü (Faz1_IslemDongusu.xaml)

Bu modül, hazırlığı tamamlanmış iş listesindeki komutların sırasıyla yapay zeka platformlarında üretilmesi, analiz edilmesi ve optimize edilmesi işlemlerini kapsar.

A. Veri Okuma ve Oturum Kontrolü (Giriş Adımları)

- **İş Listesinin Okunması (Read: Ana İş Listesi):** İşlem döngüsü başlamadan önce robot, **Config** global sözlüğünde belirtilen hedef dosya yolundaki güncel çalışma Excel'ini (WIP) okur. Okunan bu veriler, sürecin ilerleyen adımlarında güvenle kullanılmak üzere In/Out argümanı tipindeki **io_dt_IsListesi** veri tablosuna (DataTable) aktarılır.

Tarayıcı Bağlantısı ve Arayüz Taraması (Browser:Ideogram Ana Sayfa): Robot, belirlenen hedef URL'yi (in_IdeogramUrl) kullanarak Ideogram platformuna bağlanır. Sayfa yüklendiğinde, UiPath Computer Vision (CV) motoru devreye girerek ekranı tarar ve "Sign in" butonunun görünür olup olmadığını kontrol eder. Tarama sonucu mantıksal (Boolean) bir değişkene (bool_LoginButonuVarMi) kaydedilir.

- **Oturum Doğrulama ve İnsan Müdahalesi (FlowDecision: Kontrol: Giriş Butonu?):** Robot, arayüz tarama sonucunu değerlendirerek rotasını belirler:
    - **Oturum Kapalıysa (True):** Eğer ekranda "Sign in" butonu tespit edilirse, robot bir uyarı logu (Log: Giriş Yapılmamış) atarak süreci duraklatır. Kullanıcıya bir **Message Box** ("Lütfen giriş yapın ve ardından Tamam'a basın.") çıkartılır. Kullanıcı manuel girişini tamamlayıp onay verene kadar robot bekler. Onay sonrası güvenlik amacıyla tarayıcı kontrol döngüsü baştan başlatılır (Loop).
    - **Oturum Açıksa (False):** "Sign in" butonu tespit edilmezse, robot mevcut bir oturum olduğunu doğrular, sisteme başarı logunu (Log: Giriş Başarılı) yazar ve tasarımların üretileceği ana döngüye kesintisiz geçiş yapar.

B. Ana İşlem Döngüsü ve Statü Kontrolü (Main Loop & Status Validation)

- **Üretim Döngüsüne Giriş (Log: Üretim Başlatılıyor):** Oturum doğrulaması sağlandıktan sonra robot, io_dt_IsListesi tablosundaki toplam satır sayısını dinamik olarak hesaplar. Üretim sürecinin resmi olarak başladığını sisteme bildirmek için şu formatta global bir bilgi logu (Info) atarak ana işlem döngüsüne girer:

*"İş listesi başarıyla yüklendi. Toplam **[İşlenecek Satır Sayısı]** satır için üretim süreci başlatılıyor."*

- **Döngü Başlatma (FlowDecision: idx_SatirNo < io_dt_IsListesi.Rows.Count):** Oturum doğrulaması sağlandıktan sonra robot, iş listesindeki satır sayısı kadar çalışacak bir ana döngü başlatır. Süreç, sıfırdan başlayan bir sayaç değişkeni (**idx_SatirNo**) ile kontrol edilir.
- **Statü Doğrulama (FlowDecision: Q: State = ready_ceo?):** Robot, sıradaki satırı işlemeye almadan önce hafızadaki veri tablosunun (io_dt_IsListesi) o anki satırına ait "State" (Durum) sütununu kontrol eder. Yalnızca statüsü **"ready_ceo"** olan satırlar (yani Initialization aşamasında promptu başarıyla hazırlanmış ve üretime onay verilmiş olanlar) asıl üretim bandına alınır.
- **Atlama Mekanizması (Log: Atla (CEO) & Assign: İndeksi Artır):** Eğer satırın statüsü "ready_ceo" değilse (Örn: daha önceden işlem görmüş, "Done" olmuş veya "Hata" almışsa), robot o satırı işlemden muaf tutar. Sistem, durumu atladığına dair bir bilgi logu atar, sayacı bir artırır (**idx_SatirNo + 1**) ve vakit kaybetmeden bir sonraki satırı kontrol etmek üzere döngü başına (Loop) döner. Bu sayede gereksiz efor ve API maliyeti önlenir.
- **Faz 1 Kapanışı (Log: Faz1 Sonu & Kapanış):** Sayaç, tablodaki toplam satır sayısına ulaştığında ana döngü başarıyla kırılır. Robot, iş listesinin sonuna gelindiğini ve üretim fazının bittiğini loglayarak, süreç sonu raporlaması için ana State Machine üzerindeki Final State (Kapanış) evresine otonom geçiş yapar.

C. Sayfa Doğrulama ve Parametre Hazırlığı (Seq: 3 : 4 Boyut Kontrolü)

Satır statüsü onaylandıktan (ready_ceo) sonra robot, asıl üretim parametrelerini girmek üzere arayüz kontrollerini başlatır.

- **Sayfa Yüklenme ve Self-Healing (Otomatik Yönlendirme):** Robot, üretime başlamadan önce Ideogram sayfasının arayüzünün hazır olup olmadığını dinamik olarak denetler (Check App State). Eğer robot farklı bir alt sayfada kalmışsa veya ekran yüklenmemişse, olası üretim hatalarını önlemek adına sistemi anında doğru adrese (https://ideogram.ai/t/explore) yönlendirir ve durumu bir uyarı (Warn) logu ile kayıt altına alır.
- **Mevcut Boyutun (Aspect Ratio) Okunması:** Doğru sayfanın yüklendiği teyit edildikten sonra UiPath Computer Vision (CV) motoru devreye girer. Arayüzdeki güncel en-boy oranı metni (Örn: 1:1, 16:9 vb.) okunarak robotun hafızasındaki **str_MevcutBoyut** değişkenine aktarılır. Bu adımın izlenebilirliği için sisteme şu bilgi logu bırakılır:

*"Ideogram'dan okunan mevcut boyut: **[str_MevcutBoyut]**"*

D. Dinamik Boyut Yönetimi ve Veri Hazırlığı (Aspect Ratio & Data Preparation)

Arayüzden mevcut boyut bilgisi okunduktan sonra, robot bu bilginin POD (Print-On-Demand) platformu standartlarına uygunluğunu analiz eder ve üretime gönderilecek veriyi hazırlar.

- **Boyut Doğrulama Algoritması (FlowDecision: Q:Boyut 3:4 Değil mi?):** Robot, okunan değerin (str_MevcutBoyut) doğruluğunu katı bir eşleşme yerine esnek bir RegEx (Düzenli İfadeler) algoritmasıyla kontrol eder. Kural cümlesi şu şekilde çalışır:
    - Mantık: String.IsNullOrEmpty(str_MevcutBoyut) OrElse NOT System.Text.RegularExpressions.Regex.IsMatch(str_MevcutBoyut, "3\s*:\s*4")
    - İş Kuralı Karşılığı: Eğer CV motoru arayüzden hiçbir değer okuyamadıysa (Null/Empty) **VEYA** okunan değer aralarındaki olası boşluklar tolere edildiğinde dahi "3:4" formatına uymuyorsa, robot boyutun "Hatalı" (True) olduğuna karar verir.
- **Otonom Boyut Düzeltme (Seq: Boyut Ayarı Düzeltme):** * Karar mekanizmasından dönen sonuç hatalıysa (True), robot arayüzdeki boyut seçenekleri menüsünü Computer Vision (CV) tıklamasıyla açar ve listeden kesin olarak "3:4" oranını seçer. İşlemin ardından boyutun başarıyla güncellendiğini loglar.
    - Eğer boyut zaten 3:4 ise (False), robot gereksiz arayüz tıklamalarını (UI Interaction) atlayarak zamandan tasarruf eder ve sürece devam ettiğine dair bir bilgi logu (Log: Boyut Zaten 3:4) atar.
- **Üretim Verisinin Hazırlanması (Seq: Veri Hazırlık Aşaması):** Arayüz parametreleri (boyut) kesinleştirildikten sonra, robot ilgili satıra ait üretim komutunu hafızaya alır. Ana veri tablosundaki (io_dt_IsListesi) o anki satıra (idx_SatirNo) ait "Prompt" sütunundaki metin çekilerek **str_GecerliPrompt** değişkenine atanır. İşlemin izlenebilirliği için, ana şablonun (C2) atlandığını doğrudan belirten ve formül (idx_SatirNo + 2) ile o anki gerçek Excel satır numarasını hesaplayan şu bilgi logu sisteme işlenir:

"C2 atlandı, C*[İşlenen Excel Satır Numarası]** satırı işleniyor. Prompt: **[str_GecerliPrompt]**"*

E. Yapay Zeka Üretimi ve Dinamik Bekleme (Seq: Prompt Girişi ve Üretim)

Gerekli parametreler sağlandıktan sonra robot, Ideogram AI motorunu tetikler ve üretim sürecinin sonuçlanmasını otonom olarak denetler.

- **Prompt Girişi ve Tetikleme:** Robot, hafızasına aldığı tasarım komutunu (str_GecerliPrompt), Ideogram'daki metin kutusuna Computer Vision (CV) teknolojisi ile yazar. Ardından "Generate" butonuna tıklayarak üretimi başlatır ve süreci izlemek adına şu bilgi logunu sisteme bırakır:

"Üretim tetiklendi. Görsellerin oluşması bekleniyor (Max 180 sn)..."

- **Otonom Üretim Beklemesi (Check App State):** Üretim komutu verildikten sonra robot, statik bir bekleme (hard-delay) yapmak yerine arayüzde "Generation complete" ibaresinin belirmesini dinamik olarak tarar. Sistem kilitlenmelerini (deadlock) önlemek amacıyla bu denetim maksimum 180 saniye (3 dakika) ile sınırlandırılmıştır.
- **Başarılı Üretim ve Rastgele Bekleme (Target appears):** Eğer hedef ibare 180 saniye içerisinde ekranda belirirse, robot görsellerin tarayıcıya tam çözünürlükte yüklenmesini (render) garanti altına almak için sisteme **TimeSpan.FromMilliseconds(New Random().Next(3000, 7001))** formülüyle **3 ila 7 saniye arasında dinamik bir bekleme** (Delay) uygular. Ardından *"Üretim bitti. Görseller indirilmeye hazırlanıyor..."* logunu atarak indirme aşamasına geçer.
- **Zaman Aşımı ve İstisna Yönetimi (Target does not appear):** Platformun aşırı yoğun olması veya yanıt vermemesi sebebiyle hedef ibare belirtilen sürede ekrana gelmezse, robot süreci sonsuz döngüden kurtarmak için Error (Hata) seviyesinde *"Hata: 180 saniye içinde üretim tamamlanamadı. Akış durduruluyor."* logunu atar. Hemen ardından aşağıdaki sistem hatasını fırlatarak (Throw) süreci güvenli kapanış (Final State) rotasına yönlendirir:

Fırlatılan İstisna: New SystemException("Ideogram üretim süresi zaman aşımına uğradı.")

F. Hafıza Yönetimi ve Yeni Döngü Hazırlığı (Seq: Hafıza ve Liste Temizliği)

Başarılı bir üretim tetiklemesinin ardından, indirme ve analiz aşamalarına geçmeden önce robot, veri bütünlüğünü korumak ve olası dosya çakışmalarını önlemek adına geçici bellek temizliği yapar.

- **Geçici Listenin Sıfırlanması:** Robot, bir önceki döngüden veya sistem denemelerinden kalan dosya yollarının yeni tura karışmaması için **list_GorselYollari** isimli liste (List) değişkenini tamamen sıfırlar (New List(Of String)). Bu işlem, her bir tasarım için sadece o tura ait görsellerin takip edilmesini garanti altına alır.
- **İzlenebilirlik Kaydı:** Bellek temizleme işleminin başarıyla gerçekleştiğini ve sistemin yeni veri akışına (dosya indirme ve isimlendirme) hazır olduğunu bildirmek amacıyla Trace (İzleme) seviyesinde şu teknik log sisteme işlenir:

"Dosya yolu listesi temizlendi. Yeni döngü için hazır."

G. Kullanıcı Onayı ve Tasarım Seçimi (MsgBox: Manuel Görsel İndirme)

Üretim süreci tamamlandıktan sonra robot, yapay zekanın ürettiği alternatifler arasından en kaliteli olanın sisteme dahil edilmesi için süreci askıya alarak kullanıcı denetimine sunar.

- **Tasarım Seçimi ve Kalite Kontrolü:** Robot, hafıza temizliği işleminin hemen ardından ekrana bir yönlendirme kutusu (Message Box) çıkartır. Bu aşamada kullanıcı, Ideogram platformu tarafından sunulan 4 farklı tasarım alternatifini estetik ve teknik açıdan (yazım hatası, görsel netliği vb.) inceler.
- **Manuel Dosya Aktarımı:** Kullanıcı, seçtiği en yüksek kaliteli **tek bir görseli** manuel olarak yerel dizindeki **03-Gemini-Temp** klasörüne kaydeder. Robot, kullanıcı bu işlemi tamamlayıp mesaj kutusundaki onay butonuna tıklayana kadar pasif bekleme modunda kalır.
- **Bilgilendirme Mesaj Şablonu:** Kullanıcı arayüzünde gösterilen ve süreci hatasız işletmeyi amaçlayan yönlendirme metni şu şekildedir:

*"Lütfen üretilen alternatifler arasından en kaliteli tasarımı seçip **03-Gemini-Temp** klasörüne kaydedin ve ardından **Tamam** butonuna basın."*

- **Akışın Devamı:** Onay işleminin alınmasıyla birlikte robot, klasörde analiz edilmeyi bekleyen nihai görselin hazır olduğunu doğrular ve bir sonraki modül olan **Gemini SEO Analizi** aşamasını otonom olarak tetikler.

H. AI Destekli SEO Analizi ve RegEx Veri Ayıklama (Seq: Gemini Analizi ve RegEx ile Karar Çekme)

Bu aşamada robot, yerel dizine kaydedilen tasarımı Google Gemini Vision yapay zeka modeline göndererek profesyonel bir içerik analizi ve isimlendirme süreci yürütür.

- **Dizin Taraması ve Dosya Listeleme:** Robot, öncelikle kullanıcı tarafından görselin kaydedildiği geçici dizini (03-Gemini-Temp) tarar. **Directory.GetFiles** metodu kullanılarak dizindeki tüm PNG dosyaları **list_GorselYollari** değişkenine aktarılır. İşlem sonunda dizin durumu şu formatta loglanır:

*"Klasör tarama tamamlandı. Bulunan dosya sayısı: **[Dosya Sayısı]** | Dosya listesi: **[Dosya İsimleri]**"*

- **Gemini Vision ile Görsel Analizi:** Robot, dizindeki görseli (veya ilk dosyayı) Gemini Vision API'ye iletir. Gönderilen prompt (komut), yapay zekadan şu spesifik görevleri yerine getirmesini ister:
    - Görselin "Retro Groovy" tarzına uygunluğunu ve yazım hatası olup olmadığını denetlemek.
    - Tasarım üzerindeki metni temel alan, kelimelerin tire (-) ile bağlandığı ve sonunda retro-groovy ibaresi yer alan SEO uyumlu bir dosya ismi önermek.
- **Ham Yanıtın Kaydı:** Yapay zekadan gelen tüm metinsel yanıt, ileri düzey ayıklama işlemleri için **str_GeminiCevap** değişkenine atanır ve Trace logu olarak kaydedilir:

*"Gemini Ham Cevap: **[AI Yanıt Metni]**"*

- **RegEx ile Hassas Veri Çekme:** Robot, Gemini'nin verdiği uzun yanıttan sadece ihtiyacı olan dosya ismini çekmek için Düzenli İfade (RegEx) teknolojisini kullanır.
    - *Pattern (Kalıp):* [a-z0-9-]+-retro-groovy
    - Bu kalıp sayesinde robot, karmaşık cümleler arasından sadece SEO formatına uygun dosya ismini cımbızlayarak **str_SeoDosyaIsmi** değişkenine yazar.
- **Karar Mekanizması ve Hata Yönetimi (If: SEO İsmi Ayıklandı mı?):** Ayıklama işleminin başarısı kontrol edilir:
    - **Hata Durumu (Then):** Eğer RegEx sonucunda geçerli bir isim bulunamazsa (String.IsNullOrEmpty), robot "Hata: Gemini cevabından SEO ismi ayıklanamadı!" logunu atar. Süreç, hatalı isimlendirmeyi önlemek amacıyla aşağıdaki **İş Kuralı Hatası (BRE)** ile durdurulur:

Fırlatılan İstisna: New BusinessRuleException("SEO ismi bulunamadığı için süreç durduruldu.")

    - **Başarı Durumu (Else):** SEO ismi başarıyla alındığında robot, "SEO ismi başarıyla alındı: **[str_SeoDosyaIsmi]**" logunu düşerek bir sonraki adım olan dosya taşıma ve Python optimizasyon aşamasına geçer.

I. SEO Kayıt ve Dosya Transferi (Seq: SEO: İsimlendirme ve Dosya Taşıma)

Gemini tarafından belirlenen SEO ismi doğrulandıktan sonra robot, hem veritabanı (Excel) güncellemesini yapar hem de fiziksel dosyayı üretim bandında bir sonraki klasöre taşır.

- **Dosyanın SEO Adıyla Yeniden İsimlendirilmesi (Rename File):** Robot, geçici klasördeki (03-Gemini-Temp) ham dosyayı alır ve Gemini'den gelen SEO ismini (**str_SeoDosyaIsmi**) kullanarak dosyanın adını günceller. Bu işlem, e-ticaret sitelerinde görselin bulunabilirliğini artırmak için hayati önem taşır.
- **İş Listesinin (Excel) Güncellenmesi (Write Cell):** Belirlenen dosya ismi, takip ve raporlama amacıyla WIP (Work-In-Progress) Excel dosyasına anlık olarak yazılır.
    - Dosya Yolu: **Config("in_WIP_ExcelPath")**
    - Hedef Hücre: SEO isimlerinin tutulduğu sütun (**in_SeoNameColumn**) ile o anki satırın (**idx_SatirNo + 2**) birleşimiyle dinamik olarak hesaplanır.
- **İzlenebilirlik ve Takip Logları:** İşlemin şeffaflığı için şu formatta bilgi logları sisteme düşülür:

*"Şu an işlenen satır: **[Satır No]** | SEO İsmi: **[str_SeoDosyaIsmi]**" "Taşıma işlemi başlatılıyor: siradaki.png -> 04-Optimize-Process"*

- **Fiziksel Dosya Taşıma (Move File):** İsimlendirme ve kayıt işlemleri bittikten sonra görsel, analiz klasöründen çıkarılarak görsel işleme (Python) klasörüne transfer edilir.
    - Kaynak: Path.Combine(Config("in_GeminiTempPath").ToString, str_SeoDosyaIsmi + ".png")

    - Hedef: Path.Combine(Config("in_OptimizePath").ToString, str_SeoDosyaIsmi + ".png")
    - Not: Olası çakışmaları önlemek adına "Overwrite" (Üzerine Yaz) seçeneği aktif tutulur.
- **Transfer Onayı:** Taşıma işlemi bittiğinde robot, Python modülünün tetiklenebileceğine dair şu başarı logunu atar:

*"Satır **[Satır No]** dosyası 04-Optimize-Process klasörüne başarıyla taşındı. Python işlemi başlatılabilir."*

J. Statü Senkronizasyonu ve Python Ön Hazırlık (Seq: Python Hazırlık)

Robot, tasarımın fiziksel transferini tamamladıktan sonra, bir sonraki aşama olan gelişmiş görüntü işlemenin (arka plan silme, çözünürlük artırma vb.) başlayabilmesi için sistem statülerini günceller.

- **Excel Veritabanının Güncellenmesi (Write Cell):** Robot, WIP (Work-In-Progress) dosyasındaki ilgili satırın durum sütununa (**in_StatusColumn**) erişir ve buraya "ready_python" bilgisini yazar. Bu işlem, sürecin herhangi bir noktada kesilmesi durumunda robotun kaldığı yerden devam edebilmesini (durum takibi) sağlar.
    - Log Kaydı: `"Satır **[Satır No]** işlemi başarıyla tamamlandı. Durum 'ready_python' olarak işaretlendi."*
- **Canlı Bellek (DataTable) Güncellemesi (Assign):** Robot, sadece dosyayı değil, kendi çalışma hafızasını da güncel tutar. **io_dt_IsListesi** argümanındaki o anki satıra ait "State" (Durum) kolonu "ready_python" olarak set edilir.

Veri Tutarlılığı Kontrolü: İşlemin sonunda robot, hafızadaki verinin başarıyla güncellendiğini doğrulamak için **io_dt_IsListesi** tablosundaki ilgili hücreyi tekrar okur ve güncel durumu şu formatta loglar:

*"Satır **[Satır No]** işlemi tamamlandı. DataTable (io_dt_IsListesi) hafızasındaki güncel durum: **[Hücrenin Güncel Değeri]**"*

K. Gelişmiş Görüntü İşleme ve Koşullu Durum Yönetimi (Seq: Python Tasarım İşleme Modülü)

Tasarım süreci bittikten sonra görselin nihai optimizasyonu (arka plan temizleme, çözünürlük artırma vb.) hibrit bir teknoloji mimarisiyle Python motoru üzerinden gerçekleştirilir.

- **Python Çalışma Ortamı Yapılandırması (Python Scope):** Robot, harici betiklerin güvenli ve performanslı çalışması için izole bir çalışma ortamı başlatır. Bu kapsamda **Config** sözlüğünden alınan teknik parametreler (in_str_PythonDLLYolu, in_str_PythonKurulumYolu, in_str_AnaCalismaKlasoru) ile sistem kütüphaneleri doğrulanır.

- **Betik Yükleme ve Fonksiyon Çağrımı:** Hedef dizindeki Python dosyası belleğe yüklenir. Ardından, görüntü işleme algoritmalarını barındıran **tasarimi_isleme_sok** metodu, o anki dosya adı (**str_SeoDosyaIsmi**) parametre olarak verilerek tetiklenir.
- **İşlem Sonucunun Alınması:** Python motorundan dönen ham veri (py_MetotSonucu), robot tarafından okunabilir metin formatına dönüştürülür ve analiz edilmek üzere **str_PythonFinalCevap** değişkenine aktarılır.
- **Koşullu Durum Analizi (If: Python İşlemi Başarılı mı?):** Robot, Python'dan gelen yanıtı bir karar mekanizması (If) üzerinden değerlendirerek süreci iki farklı rotaya yönlendirir:
    - **Başarı Senaryosu (Then):** Eğer dönen yanıt kesin olarak **"SUCCESS"** ise; robot işlemin kusursuz tamamlandığına karar verir. Bu aşamada Excel'deki ilgili satırın durum sütununa (**in_StatusColumn**) kalıcı olarak **"Done"** bilgisi yazılır ve işlem başarıyla kapatılır.
    - **Hata Senaryosu (Else):** Eğer Python motorundan "SUCCESS" dışında bir yanıt (hata kodu veya sistem uyarısı) gelirse; robot bu hatayı yakalar. Kullanıcının süreci kontrol edebilmesi adına, dönen gerçek hata mesajı ("Hata: " + str_PythonFinalCevap) doğrudan Excel'deki ilgili hücreye işlenir.
- **Sistem Temizliği ve Kaynak Yönetimi (Kill Process):** Her bir tasarım döngüsünün sonunda robot, sistem kaynaklarını (RAM/CPU) optimize etmek ve tarayıcı şişmelerini önlemek adına arka planda çalışan Chrome oturumlarını otonom olarak sonlandırır.
- **Döngü Başına Dönüş:** Mevcut satırın statüsü ("Done" veya "Error") belirlendikten sonra robot, iş akışını otomatik olarak Faz 1'in başlangıç noktasına, yani **Log: Üretim Başlatılıyor** adımına geri döndürür.

7.3. Finalize_Workflow: Süreç Kapanış ve Raporlama (Finalize_Workflow.xaml)

Bu modül; üretim döngüsü tamamlandıktan veya süreç herhangi bir aşamada kesintiye uğradıktan sonra tetiklenen; hata analizi, bildirim yönetimi ve sistem temizliği işlemlerini kapsayan final aşamasıdır.

A. Kapanış Başlangıcı ve Durum Analizi (Log: Final Başladı & If: Hata Kontrolü)

Robot, kapanış evresine geçtiğinde ilk olarak mevcut çalışma durumunu analiz ederek rotasını belirler.

- **Final İşlemleri Başlangıç Kaydı:** Otomasyonun üretim aşamasından çıkıp raporlama aşamasına girdiğini belgelemek adına sisteme şu bilgi logu düşülür:

"Final işlemleri başladı"

- **Kritik Hata Kontrol Mekanizması (If: Hata Kontrolü):** Robotun ana akışı bir hata nedeniyle mi yoksa başarıyla mı tamamladığı, süreç boyunca toplanan istisna (exception) verileri üzerinden denetlenir.
    - **Mantıksal Koşul:** in_SystemException IsNot Nothing Or in_BusinessException IsNot Nothing
    - **Başarı Senaryosu (False):** Süreç boyunca herhangi bir teknik aksaklık veya iş kuralı hatası fırlatılmamışsa, robot **False** dalını takip ederek sırasıyla şu modülleri çalıştırır:
        - **Seq: Başarı Mesajı Gönder**: Sürecin kusursuz bittiğine dair başarı raporunu ilgili paydaşlara iletir.
        - **Seq: Süreç Sonu ve Arşiv İşlemleri**: Çıktıları nihai klasörlerine taşır, geçici dosyaları temizler ve aktif oturumları sonlandırarak süreci güvenli bir şekilde kapatır.
    - **Hata Senaryosu (True):** Eğer bir hata tespit edilirse, robot **True** dalına saparak hata yönetim ve analiz sürecini başlatır.

B. Akıllı Hata Çözümleme ve Doğal Dil Dönüşümü (Seq: Gemini AI Hata Analizi)

Süreç içerisindeki **If: Hata Kontrolü** mekanizması **True** sonucunu verdiğinde (yani sistemde teknik bir aksaklık veya iş kuralı ihlali tespit edildiğinde), robot bu özel analiz modülünü tetikler. Buradaki temel amaç; yazılım dünyasına ait karmaşık hata mesajlarını, teknik bilgisi olmayan son kullanıcının anlayabileceği şeffaf ve çözüm odaklı bir bilgilendirme metnine dönüştürmektir.

1. Hata Tespit ve Başlatma Mantığı

Robot, analiz sürecine başlamadan önce karşılaşılan hatanın türünü belirleyerek sisteme bir ön rapor sunar. Bu aşamada kullanılan dinamik loglama yapısı, hatanın kaynağını (Sistem veya İş) anında ayırt eder:

Analiz Başlangıç Kaydı: "Gemini AI çözümleme süreci başlatıldı. Analiz edilen hata: " + If(in_SystemException IsNot Nothing, "[Sistem Hatası] " + in_SystemException.Message, If(in_BusinessException IsNot Nothing, "[İş Hatası] " + in_BusinessException.Message, "Tanımlanamayan bir hata oluştu."))

2. Gemini AI: "RPA Asistanı" Rolü ve Prompt Stratejisi

Hata mesajı, gemini-2.5-flash modeline özel bir "istatistiksel sadeleştirme" komutuyla (Prompt) iletilir. Bu komutun merkezinde, teknik jargonu tamamen temizleyerek hatayı insani bir dille özetlemek yatar.

Prompt Yapısı ve İşleyişi: Yapay zekaya yüklenen **"RPA Asistanı"** kimliğiyle şu kurallar çerçevesinde bir analiz istenir:

- **Neden-Sonuç İlişkisi:** Mesajın her zaman **"... durumundan dolayı ... işlemi gerçekleştirilemedi"** mantığıyla kurulması sağlanır. Bu sayede kullanıcı, hatanın kök nedenini (örneğin; dosya eksikliği, web sitesinin yanıt vermemesi veya verilerdeki eksiklikler) net bir şekilde kavrar.
- **Teknik Terim Temizliği:** Kullanıcıyı "NullReference", "Exception" veya "SelectorNotFound" gibi karmaşık terimlerle boğmak yerine, sorunun günlük dildeki karşılığı verilir.
- **UX Standartları:** Çıktının son derece kibar, Türkçe dil kurallarına uygun ve en fazla 2-3 cümlelik öz bir bilgilendirme olması zorunlu tutulur.

3. Çıktı ve Karar Mekanizması

Analiz sonucunda Gemini tarafından hazırlanan bu kullanıcı dostu özet, **str_FinalGeminiHataMesaji** değişkenine aktarılır. Bu aşama tamamlandığında robot, hazırlanan nihai mesajı doğrulamak adına şu teknik kaydı düşer:

Analiz Onay Kaydı: "Gemini analizi tamamladı. Üretilen mesaj: " + str_FinalGeminiHataMesaji

C. Hata Bildirimi ve Otomatik E-Posta Gönderimi (Seq: Hata Mesajı Gönder)

Hata analizi modülü başarıyla tamamlandıktan sonra robot, hazırlanan sadeleştirilmiş raporu ilgili paydaşlara iletmek üzere Gmail entegrasyonunu kullanır. Bu sayede süreçteki aksamalar anlık olarak raporlanır.

1. Gönderim Hazırlığı ve İzlenebilirlik

Robot, e-posta gönderim işlemine başlamadan önce alıcı ve konu bilgilerini içeren teknik bir kayıt (Log) oluşturur. Bu adım, sürecin hangi adrese ve hangi başlıkla raporlandığını denetlemek için kullanılır:

Hazırlık Logu: "Hata bildirim e-postası gönderiliyor... Alıcı: " + Config("in_str_AliciMailAdresi").ToString + " | Konu: " + Config("in_str_MailKonusu").ToString

2. Akıllı Bilgilendirme E-Postası (Gmail Entegrasyonu)

Robot, Google Integration Service üzerinden önceden yetkilendirilmiş bir Gmail oturumu ile bağlantı kurar. E-posta içeriği, statik metinler yerine tamamen **Config** dosyasından ve **Gemini** analizinden beslenen dinamik bir yapıda kurgulanmıştır:

- **Alıcı Yönetimi:** E-posta, Config dosyasında tanımlı olan **in_str_AliciMailAdresi** değişkenindeki adrese yönlendirilir.
- **Dinamik Konu Başlığı:** Mesajın konusu, yine merkezi ayarlardan çekilen **in_str_MailKonusu** verisiyle oluşturulur.
- **Sadeleştirilmiş İçerik (Body):** E-postanın gövde kısmında, bir önceki adımda Gemini tarafından teknik terimlerden arındırılmış olan **str_FinalGeminiHataMesaji** değişkeni yer alır. Bu sayede alıcı, teknik bir hata kodu yerine "İnternet bağlantısı koptuğu için işlem tamamlanamadı" gibi net bir açıklamayla karşılaşır.

3. Gönderim Onayı

İletim işlemi hatasız bir şekilde tamamlandığında, robot sürecin başarıyla sonuçlandığını doğrulamak adına şu bilgi kaydını düşer:

Onay Logu: "Hata bildirim e-postası başarıyla gönderildi."

Hata bildiriminin gönderilmesiyle birlikte robot, akış diyagramındaki mantıksal yolunu tamamlar ve otonom olarak **Seq: Süreç Sonu ve Arşiv İşlemleri** modülüne geçiş yapar.

D. Başarı Bildirimi ve Süreç Tamamlama (Seq: Başarı Mesajı Gönder)

If: Hata Kontrolü mekanizması **False** sonucunu verdiğinde (yani tüm iş listesi hatasız bir şekilde "Done" statüsüne ulaştığında), robot bu modülü tetikleyerek paydaşlara operasyonun başarıyla tamamlandığını raporlar.

- **Hazırlık ve İzlenebilirlik:** Robot, bildirim sürecini başlatmadan önce alıcı bilgisini içeren bir kayıt (Log) oluşturarak şeffaflığı sağlar:

Hazırlık Logu: "Başarı bildirim e-postası hazırlanıyor... Alıcı: " + Config("in_str_AliciMailAdresi").ToString

- **Gmail Başarı Raporu:** Google Integration Service aracılığıyla, merkezi **Config** dosyasında tanımlanan parametreler doğrultusunda bir e-posta gönderilir:
    - **Alıcı:** Config("in_str_AliciMailAdresi") değişkenindeki adrese yönlendirilir.
    - **Konu:** Projenin genel başarı başlığını içeren Config("in_str_MailKonusu") kullanılır.
    - **İçerik (Body):** Kullanıcıya sürecin bittiğini müjdeleyen ve yine Config dosyasından beslenen **in_str_BasariliIslemMesaji** içeriği iletilir.
- **Gönderim Onayı:** E-posta iletimi sorunsuz bittiğinde robot şu teknik onayı sisteme işler:

Onay Logu: "Log: Başarı bildirim e-postası sorunsuz gönderildi. Alıcı: " + Config("in_str_AliciMailAdresi").ToString

Başarı bildiriminin gönderilmesiyle birlikte robot, akış diyagramındaki mantıksal yolunu tamamlar ve otonom olarak **Seq: Süreç Sonu ve Arşiv İşlemleri** modülüne geçiş yapar.

E. Süreç Sonu ve Arşiv Yönetimi (Seq: Süreç Sonu ve Arşiv İşlemleri)

Robotun hem hata (True) hem de başarı (False) yollarının sonunda ulaştığı bu ortak modül; çalışma süresince güncellenen verilerin kalıcılığını sağlamak ve geriye dönük izlenebilirlik (loglama) oluşturmak amacıyla kurgulanmıştır.

1. Arşivleme Başlangıcı ve Konum Bilgisi

Robot, sistem kaynaklarını kapatmadan önce çalışma dosyasının güvenli bir konuma aktarılacağını sisteme bildirir. Bu adım, özellikle çoklu çalıştırmalarda hangi hedef klasörün kullanıldığını doğrulamak için kritiktir:

Başlatma Logu: "Arşivleme süreci başlatılıyor. Hedef klasör: " + Config("in_str_ArsivKlasorYolu").ToString

2. Veri Kalıcılığı ve Tarihsel Yedekleme (Copy File)

Robot, üzerinde çalıştığı güncel "WIP" (Work-In-Progress) Excel dosyasını, merkezi arşiv dizinine kopyalar. Burada kullanılan dinamik isimlendirme stratejisi, manuel takip yükünü ortadan kaldırır:

- **Kaynak Dosya:** Config("in_WIP_ExcelPath") üzerinden mevcut çalışma listesi baz alınır.
- **Dinamik İsimlendirme:** Dosya, hedef klasöre aktarılırken Now.ToString("dd-MM-yyyy") formülüyle o günün tarihiyle mühürlenir. (Örn: *Is-Listesi-19-03-2026.xlsx*)
- **Arşiv Mantığı:** Path.Combine metodu kullanılarak dosya yolu ve yeni isim hatasız bir şekilde birleştirilir. Bu sayede her iş günü için ayrı bir rapor dosyası oluşturularak tarihsel bir veri bankası inşa edilir.

3. Final Onayı ve Süreç Kapanışı

Kopyalama işlemi başarıyla tamamlandığında robot, tüm operasyonun resmen bittiğini ve arşiv raporunun hazır olduğunu duyuran final logunu sisteme işler:

Tamamlandı Logu: "Süreç tamamlandı. Arşiv raporu oluşturuldu: Is-Listesi-" + Now.ToString("dd-MM-yyyy") + ".xlsx"

F. Sistem Temizliği ve Operasyonel Kapanış (Kill Process & Final Log)

İş akışının tüm analiz, raporlama ve arşivleme adımları tamamlandıktan sonra robot, sistem kaynaklarını serbest bırakmak ve temiz bir çalışma ortamı bırakmak adına son işlemleri gerçekleştirir.

- **Uygulama Sonlandırma (Kill Process: Chrome):** Robot, süreç boyunca kullanılan Google Chrome tarayıcı oturumlarını otonom olarak sonlandırır. Bu adım, hem sistem belleğinin (RAM) gereksiz tüketilmesini önlemek hem de bir sonraki çalışmada oluşabilecek oturum çakışmalarının önüne geçmek amacıyla kurgulanmıştır.
- **Süreç Sonu Onay Kaydı (Log: Başarı ile Sonlandırma):** Operasyonun tüm adımlarının (Init, Faz 1 ve Finalize) sorunsuz bir şekilde sona erdiğini belgelemek amacıyla sisteme şu final bilgi logu (Info) işlenir:

Final Mesajı: "Kapanış süreci başarıyla tamamlandı ve tüm kaynaklar temizlendi."

8. Hata ve İstisna Yönetimi (Exception Handling)

Süreç, robotun beklenmedik durumlarda kontrolsüz kapanmasını önlemek ve her hatayı analiz edilebilir bir veri olarak kaydetmek amacıyla **State Machine** mimarisi üzerine kurgulanmıştır.

A. Global İstisna Yakalama Mekanizması

Sürecin her ana evresi (Initialization, Faz 1 vb.) **Try/Catch** blokları ile sarmalanmıştır. Bir hata meydana geldiğinde robot, akışı kesmek yerine hatayı türüne göre sınıflandırarak global değişkenlere kaydeder:

- **SystemException:** Web sitesinin yanıt vermemesi veya sistem kaynaklı teknik aksaklıklar bu değişkene aktarılır.
- **BusinessException:** Taslak promptlarda etiket eksikliği veya iş kuralı ihlalleri gibi mantıksal hatalar bu değişkene atanır.

B. Kritik Müdahale ve Throw Kullanımı

Robot, sürecin devam etmesinin hatalı sonuç doğuracağı veya veri bütünlüğünü bozacağı noktalarda **Throw** aktivitesini kullanarak akışı bilinçli bir şekilde durdurur Örneğin;

- **Üretim Zaman Aşımı:** Ideogram üzerinde görsel üretimi belirlenen 180 saniyelik sınırı aşarsa, robot bir sistem hatası fırlatarak otonom olarak kapanış rotasına girer .
- **SEO Veri Doğrulama:** Gemini AI tarafından üretilen dosya isminin RegEx ile ayıklanamaması durumunda, hatalı isimlendirmeyi önlemek adına bir iş kuralı hatası (BRE) fırlatılır .

C. Hata Sonrası Durum Geçişleri (Transitions)

Yakalanan tüm istisnalar (Exception), robotu State Machine üzerindeki **T3** geçiş yoluna yönlendirir. Bu yol üzerinden doğrudan **Final State** (Kapanış) evresine atlayan robot, hafızasındaki hata mesajını Gemini AI ile analiz ettirerek kullanıcıya raporlar ve sistemi güvenli bir şekilde kapatır.

9. Süreç İzlenebilirliği ve Loglama Stratejisi (Logging & Traceability)

Robotun çalışma anındaki performansını takip etmek, olası hataları geriye dönük analiz edebilmek ve iş birimlerine şeffaf bir rapor sunabilmek amacıyla süreç boyunca kapsamlı bir loglama stratejisi uygulanmıştır.

A. Stratejik Dar Boğaz ve Kontrol Noktası Loglaması (Checkpoint Logging)

Robot, özellikle hata riskinin yüksek olduğu veya işlem süresinin uzadığı "dar boğaz" noktalarında stratejik kontrol logları kullanır. Bu loglar; **yapılan işlem**, **güncel durum** ve **sürecin neresinde olunduğu** bilgilerini içererek izlenebilirliği artırır örneğin;

- **Veri Hazırlık Kontrolü:** "C2 atlandı, C [idx_SatirNo + 2] satırı işleniyor." mesajı ile döngünün hangi fiziksel Excel satırında olduğu netleştirilir.
- **Dış Servis Bekleme Durumu:** Yapay zeka üretim aşamasında "Üretim tetiklendi. Görsellerin oluşması bekleniyor (Max 180 sn)..." ifadesiyle robotun kilitlenmediği, aktif olarak yanıt beklediği belirtilir.
- **Modüller Arası Geçiş:** "Satır [Satır No] dosyası 04-Optimize-Process klasörüne başarıyla taşındı. Python işlemi başlatılabilir." logu, RPA modülünün görevini bitirip bayrağı Python motoruna devrettiği kritik sınırı işaretler.
- **Hata Analiz Başlangıcı:** "Gemini AI çözümleme süreci başlatıldı." kaydı ile robotun durma nedenini anlamlandırmak üzere "analiz moduna" geçtiği belgelenir.

B. Log Seviyeleri ve Kullanım Amaçları

Robot, operasyonun kritiklik derecesine göre farklı log seviyeleri kullanarak sistem yöneticilerine hiyerarşik bir takip imkanı sunar:

- **Info (Bilgi):** Sürecin ana duraklarını ve başarıyla tamamlanan aşamaları belgeler (Örn: "İş listesi başarıyla yüklendi", "Kapanış süreci tamamlandı").
- **Warn (Uyarı):** Akışı bozmayan ancak dikkat gerektiren sapmaları belirtir (Örn: "Oturum açılmamış, kullanıcı girişi bekleniyor").
- **Error (Hata):** Sürecin durmasına neden olan istisnaları ve throw mekanizmalarını kayıt altına alır.
- **Trace (İzleme):** Gemini ham yanıtları veya bellek temizliği gibi teknik detayları geliştiriciler için saklar.

C. Arşivleme ve Geriye Dönük Raporlama

Süreç sonunda robot, sadece anlık loglar üretmekle kalmaz; tüm bu izlenebilirliği o günün tarihini taşıyan bir arşiv dosyasına (Is-Listesi-dd-MM-yyyy.xlsx) mühürleyerek fiziksel bir "İşlem Kanıtı" (Evidence of Execution) oluşturur.

10. Veri Sözlüğü ve Parametre Yönetimi (Data Dictionary & Parameters)

Süreçteki veri akışı; modüller arası köprü görevi gören argümanlar (Arguments) ve çalışma anındaki durumu yöneten değişkenler (Variables) üzerinden, kurumsal isimlendirme standartlarına uygun olarak yürütülür.

A. Kritik Argümanlar ve Modüler Veri Transferi

Modüller (.xaml dosyaları) arasındaki veri trafiği, UiPath standartlarına uygun yönlü argümanlar üzerinden sağlanır. Bu yapı, verinin Initialization aşamasından Finalize aşamasına kadar güvenle taşınmasını garanti eder.

Argüman İsmi	Yön	Veri Tipi	Fonksiyonel Görevi
Config	In	Dictionary	Tüm sistem ayarlarını ve API parametrelerini barındıran ana referans sözlüğüdür.
io_dt_IsListesi	InOut	DataTable	Excel'den okunan ve süreç boyunca statüleri ("Done", "ready_python" vb.) güncellenerek taşınan ana tablodur.
in_SystemException	In	Exception	Teknik hataları, Gemini analizi için Finalize_Workflow modülüne aktarır.
in_BusinessException	In	BusinessRuleException	İş kuralı ihlallerini raporlama modülüne ileten istisna nesnesidir.
in_str_AliciMailAdresi	In	String	Raporların ve hata bildirimlerinin iletileceği hedef e-posta adresidir.

B. İsimlendirme Standartları (Naming Conventions)

Kodun okunabilirliğini ve sürdürülebilirliğini sağlamak amacıyla hem argümanlarda hem de değişkenlerde ön ek (Prefix) kullanımına dayalı isimlendirme standartları uygulanmıştır.

1. Argüman İsimlendirme Kuralları:

- **in_**: Dışarıdan modül içerisine gelen statik veriler (Örn: in_ConfigFilePath, in_WipPath).
- **out_**: Modül içerisinde üretilip dışarıya aktarılan veriler.
- **io_**: Hem girdi olarak alınan hem de güncellenerek dışarıya aktarılan dinamik veriler (Örn: io_dt_IsListesi).

2. Değişken İsimlendirme Kuralları (Prefixes):

- **str_**: Metinsel verileri temsil eder (Örn: str_YeniPrompt, str_SeoDosyaIsmi).
- **bool_**: Karar mekanizmalarında kullanılan mantıksal veriler

py_: Python motoru nesneleri ve metot sonuçları

- **idx_**: Döngü sayaçları ve indeks takipleri (Örn: idx_SatirNo).
- **dt_**: Tablo formatındaki veriler (Örn: dt_Config, dt_IsListesi).